5-Day Execution Plan

AI-Generated Content Detector — Android + FastAPI Demo for AI Congress 2026 Buildfest

---

Governing principle: the judges score a live, convincing end-to-end demo, not model accuracy. Every day below is sequenced so that the app is **demoable from Day 1 onward** (mocked first, real second) and so that a slipped task on any day degrades gracefully instead of blocking the pipeline. "Smoke-and-mirrors" call-outs mark where a heuristic or a hardcoded response is deliberately substituted for real ML, because in a 5-day window the Share Sheet UX and the Red/Green verdict screen are the parts judges actually see and touch.

Android client	**Native Java**, not React Native	The whole client-side UX hinges on Android's ACTION_SEND / SEND_MULTIPLE intent filters and an overlay (chathead) Service — both are thin, well-documented native APIs. RN adds a bridge, a build-tooling tax, and a shakier overlay/foreground-service story. Native Java is faster to ship correctly in 5 days.
Backend	FastAPI + Uvicorn, deployed on Railway/Render (free tier)	Matches team's existing skillset (per stack). One deployable service, instant HTTPS URL, no infra overhead.
Visual inference	Pretrained deepfake classifier (HF hub) wrapped in heuristic fallback	Fine-tuning in 5 days is out of scope; a pretrained checkpoint + confidence threshold is enough to be real and demoable.
Text/claim check	OCR (Tesseract) + fuzzy match against a curated hoax CSV	A lookup table of 40–60 known Bangladeshi hoax phrases beats an undertrained NLP model for demo reliability.

API Contract — locked on Day 1, never renegotiated

`/verify/media`	POST (multipart)	Accepts image/video file → returns job_id
`/verify/text`	POST (json)	Accepts raw text or OCR'd claim → returns job_id
`/result/{job_id}`	GET	Polled by app every 1.5s → verdict, confidence, bn/en explanation
`/health`	GET	Uptime check for the demo laptop/judge Wi-Fi

Locking this contract on Day 1 lets frontend and backend work in parallel for the remaining 4 days against a shared mock, instead of serializing the work.

DAY 1 — Skeleton, Contract & Fake End-to-End Path

Goal: by end of Day 1, tapping Share → "our app" produces a Red or Green screen — fully mocked, zero real inference.

Track	Concrete deliverables
Frontend	• New Android Studio project (Java, min SDK 24) • AndroidManifest: activity with intent-filter for ACTION_SEND (image/*, video/*, text/plain) • Minimal "Verdict" Activity: loading spinner → Red/Green full-screen result (hardcoded data) • Retrofit/OkHttp client scaffolded against the locked API contract

Backend	• FastAPI project skeleton; implement all 4 contract endpoints returning static/mocked JSON • /verify/media and /verify/text immediately return a fixed job_id • /result/{job_id} returns a hardcoded verdict (alternate red/green for demo variety) • Deploy skeleton to Railway/Render — get a public HTTPS URL working from a phone on mobile data
Integration	• Point the Android app at the deployed mock URL (not localhost) from hour one • Verify: share an image from WhatsApp/Gallery → app calls API → shows verdict screen
RISK / MITIGATION	Biggest Day-1 failure mode is Android Share Sheet registration quirks (app not appearing in the sheet, or receiving the wrong MIME type). Budget the first 2 hours for this specifically and test on a real device, not just an emulator, since share-sheet behavior differs.

DAY 2 — Visual Pipeline — Deepfake Signal

Goal: /verify/media stops being fully mocked and returns a real (if simple) confidence score for at least one image and one video.

Track	Concrete deliverables
Backend — Image path	• OpenCV: load image, basic preprocessing (resize, face-crop if face detected via Haar cascade/MediaPipe) • Load a pretrained deepfake/AI-image detector checkpoint from Hugging Face (e.g. a ViT-based synthetic-image classifier) with PyTorch — inference only, no training • Wrap output as confidence score 0–1; threshold at 0.5 → Fake/Real
Backend — Video path	• OpenCV VideoCapture: extract 5–8 evenly-spaced frames • Run each frame through the same image classifier; aggregate via mean/max confidence • SMOKE & MIRRORS: if inference proves too slow or unstable, wrap it with a heuristic fallback (e.g. compression-artifact / frame-consistency scoring) that always returns a plausible number, so the demo never hangs or crashes on stage
Integration	• Wire /verify/media to the real pipeline; /result/{job_id} now returns real confidence + verdict • Cap processing time server-side (e.g. 8s timeout → fallback verdict) so the Android spinner never spins indefinitely during the pitch
RISK / MITIGATION	Model download/inference could blow the day. Pin a small, CPU-friendly checkpoint (not a multi-GB model) and test inference latency by mid-morning — if it exceeds ~4s per item, switch to the heuristic fallback immediately rather than debugging further.

DAY 3 — Text / Claim Pipeline — OCR + Hoax Matching

Goal: /verify/text works for both pasted text and text embedded in a shared screenshot.

Track	Concrete deliverables
Backend — OCR	• Tesseract (pytesseract) or EasyOCR for Bengali+English screenshots shared as images • Text-cleaning: strip UI chrome (timestamps, usernames) with simple regex heuristics
Backend — Claim check	• Curate a CSV/JSON of 40–60 known Bangladeshi hoax claims/headlines (bilingual) — this IS the "verified hoax database" for the demo • Fuzzy match (rapidfuzz / sentence-embedding cosine similarity via a small multilingual model, e.g. LaBSE or paraphrase-multilingual-MiniLM) against the curated set • Score above threshold → Red (matches known hoax); no match → Green with lower confidence label ("No known hoax match — verify independently") rather than a false-confidence Green
Integration	• Wire /verify/text end-to-end: raw text OR OCR'd image → match → verdict • Bilingual explanation string generated server-side (bn + en) and returned in the payload, not hardcoded client-side, so it's genuinely dynamic for the demo

RISK / MITIGATION	Bengali OCR accuracy is historically weak. Pre-select 3–4 known-good demo screenshots with clean, large text as the pitch's "reliable path," while still allowing live audience input as a stretch goal.

DAY 4 — Android UI Polish, Bilingual Layer & Fallback Chathead

Goal: the part judges actually watch — Share Sheet → verdict — looks and feels like a finished product.

Track	Concrete deliverables
Frontend — Verdict UI	• Full-screen Red/Green result: large icon (X / check), verdict word in Bengali + English stacked, one-line explanation, confidence bar • strings.xml + strings-bn/values-bn for full bilingual support; language auto-follows device locale with manual toggle as backup • Loading state with a short rotating tip ("Checking against known hoaxes…") to mask API latency
Frontend — Fallback path	• Chathead/floating-bubble Service (SYSTEM_ALERT_WINDOW) as the secondary entry point for private FB posts: tap bubble → screenshot capture (MediaProjection) → sends to /verify/media • SMOKE & MIRRORS: keep this path minimal — a single working demo case is enough; do not over-invest since Share Sheet is the primary, judged flow
Polish	• App icon, splash screen, consistent color system (Red #C0392B / Green #1E8449) matching the verdict screens • Error states: no internet, API timeout, unsupported file type — each with a friendly bilingual message, never a raw crash
RISK / MITIGATION	SYSTEM_ALERT_WINDOW permission + MediaProjection is genuinely fiddly and Android-version-dependent. Timebox it to half a day; if it isn't stable by early afternoon, cut it from the live demo and mention it verbally as "in progress" — the Share Sheet path alone proves the concept.

DAY 5 — Integration Freeze, Rehearsal & Submission

Goal: no new features. Only stability, and a pitch that cannot fail on stage.

Track	Concrete deliverables
Stabilization	• Feature freeze by 12:00 — remaining time is bug-fix and rehearsal only • Re-deploy backend to production URL; hardcode that URL in the release build (no localhost/ngrok dependency during the pitch) • Pre-warm the model/server (send a dummy request before walking on stage) to avoid cold-start lag
Demo script	• Script exactly 2 flows: (1) share a known-fake video from Gallery → Red verdict; (2) share a known-hoax text screenshot → Red verdict; keep one Green case ready as a control • Record a backup screen-capture video of both flows succeeding, in case of live Wi-Fi/venue failure — play this if live demo breaks
	• Package APK + short README (architecture diagram, business model, roadmap beyond the demo) • Submit pitch deck referencing the 50 BDT/month family safety bundle and target audience
RISK / MITIGATION	Never demo against a laptop's local server over conference Wi-Fi. Deploy properly and test the exact production URL from a phone on cellular data the night before, not the morning of.

If Time Runs Short: Cut in This Order

- 1. Chathead/screen-capture fallback (Day 4) — cut first, mention verbally as roadmap
- 2. Real deepfake model (Day 2) — fall back to the heuristic scorer; still returns a live, varying confidence number
- 3. Bengali OCR on arbitrary screenshots (Day 3) — narrow to the 3–4 pre-tested demo images only
- 4. UI polish pass (Day 4) — keep bilingual verdict + color system, drop animations/splash screen
- 5. Never cut: the Share Sheet entry point and the Red/Green verdict screen — this is the entire proof of concept

Post-Demo Roadmap (mention in pitch, do not build)

- Perceptual hashing cache layer for viral-video dedup (cost optimization, explicitly deferred per scope)
- Fine-tuned deepfake classifier on Bangladeshi-context media
- Expanded, crowdsourced hoax database with moderation pipeline
- Payment integration for the 50 BDT/month family safety bundle